
推理服务网关详细设计
KUBERNETES 原生模型服务管理方案

目录

01 项目概述与目标

解决什么问题？明确项目的核心职责与设计目标。

02 核心架构设计

解析系统整体架构拓扑与关键技术栈的选型思路。

03 核心API定义 (CRD)

详解 InferenceService 核心资源的字段定义与规范。

04 核心控制器逻辑

深入分析 Reconcile 循环机制与资源协调过程。

05 关键功能实现

介绍灰度发布策略与基于负载的自动弹性伸缩方案。

06 项目结构与代码

展示代码工程化组织方式与核心模块的关键实现。

07 部署与测试

讲解服务的环境配置、部署流程与功能验证方法。

08 总结与展望

回顾项目核心成果，并对未来的迭代方向进行规划。

项目概述与目标

当前面临的挑战

部署流程繁琐复杂

手动部署到 K8s 集群步骤多，配置易出错，效率低下。

多版本管理混乱

新旧模型版本并存，流量路由规则不清晰，回滚困难。

服务弹性伸缩不足

无法根据业务负载自动调整实例数，高峰时容易引发 OOM。

系统核心职责

统一服务入口：为算法团队提供标准化的模型部署与访问入口。

模型版本控制：支持同一模型多版本并行部署，互不干扰。

流量灰度切分：实现流量按比例平滑切换，降低上线风险。

智能弹性伸缩：基于 CPU/QPS 等指标自动扩缩容，降本增效。

2. 核心架构设计



云原生架构底座

遵循 K8s 声明式 API 与
控制器模式核心设计理念

整体架构逻辑

自定义资源 CRD

用户声明期望的模型服务状态，定义业务资源对象

控制循环组件

监听资源变化，自动创建/管理 Deployment 等原生资源

统一流量入口

基于 Ingress 实现统一接入、灰度发布与路由分发

核心技术栈选型

Kubebuilder 框架

快速构建 Kubernetes API 与控制器的脚手架

CRD 资源扩展

扩展 K8s API，定义业务专属的 InferenceService

Controller-Runtime

提供通用的控制器运行时库，简化开发流程

Ingress-Nginx

高性能流量入口，支持复杂路由与灰度策略

3. 核心API定义 (CRD)

核心 Spec 结构定义 (Go)

```
type InferenceServiceSpec struct {
ModelName string // 模型名称 | Image string // 推理镜像
Replicas int32 // 初始副本 | Resources // CPU/Memory配置
Autoscaling *AutoSpec // 弹性伸缩 | Canary *CanarySpec // 灰度
}
```

关键字段核心解析

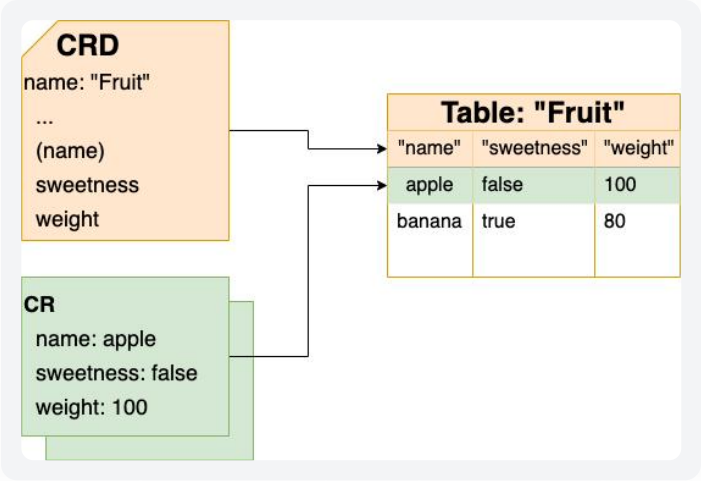
模型基础信息
定义 ModelName/Version 及推理镜像，明确部署对象。

智能弹性伸缩
配置 HPA 最小/大副本与目标利用率，应对流量波动。

资源规格限制
指定 Pod 的 CPU/内存 Requests/Limits，保障稳定性。

发布与暴露
支持 Canary 灰度权重，配置 Service 类型与域名。

Kubernetes CRD 架构逻辑



CRD 作为“蓝图”，将业务定义转化为 K8s 可识别的资源对象，实现声明式管理。

4. 核心控制器逻辑

Reconcile 核心执行循环

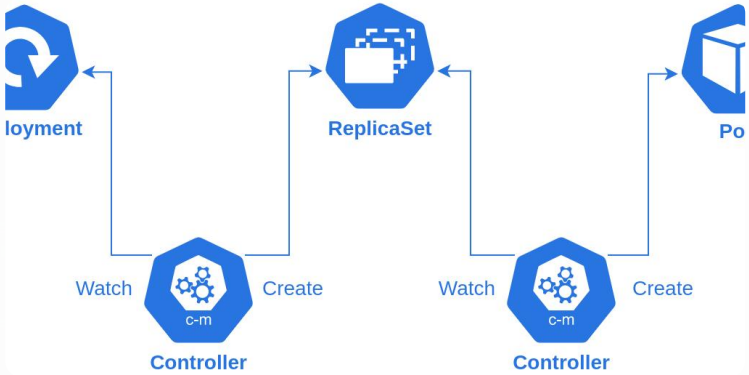
获取实例状态
拉取资源最新Spec/Status

流量与伸缩
配置Ingress路由与HPA

协调工作负载
创建/更新 稳定版与灰度版

更新资源状态
回写就绪副本与访问地址

Kubernetes 控制器协调架构



关键设计技术特性

OwnerReference
建立资源父子级联关系

统一 Labels 管理
资源追踪与分组筛选

“观察-对比-行动” 闭环
控制器通过 Watch 机制感知变化，对比 Spec 期望状态与实际状态，驱动资源向期望状态收敛，实现最终一致性。

5. 关键功能实现：灰度发布

实现原理 (Ingress-Nginx 权重路由)

01. 独立部署:创建 Stable(稳定版) 与 Canary(灰度版) 两套独立 Deployment。
02. 服务暴露:分别创建对应 Service，实现 Pod 组的负载均衡与端口暴露。
03. 流量切分:利用 Canary Ingress 注解(canary-weight)动态配置流量权重比例。

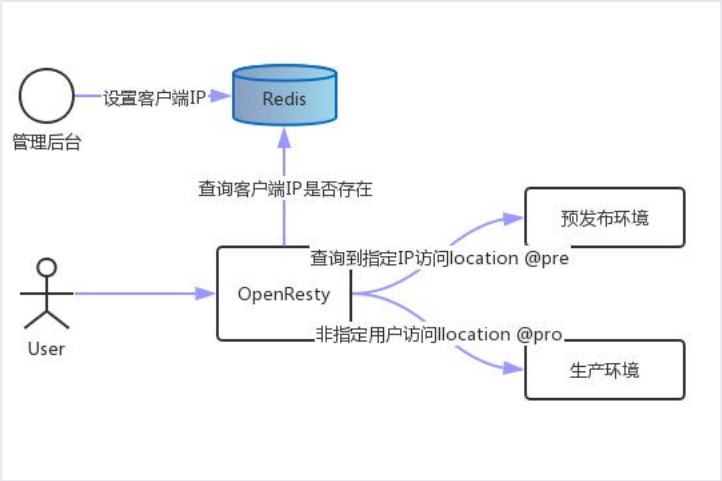
核心优势

无侵入
不改一行代码
业务无感接入

灵活控
注解动态调整
流量秒级生效

秒回滚
一键删除Ingress
快速恢复业务

流量调度架构示意



图示：基于 OpenResty 与 Redis 的客户端 IP 定向路由策略
（灰度发布流量分发核心逻辑）

5. 关键功能实现：弹性伸缩

实现原理 (HPA Controller)

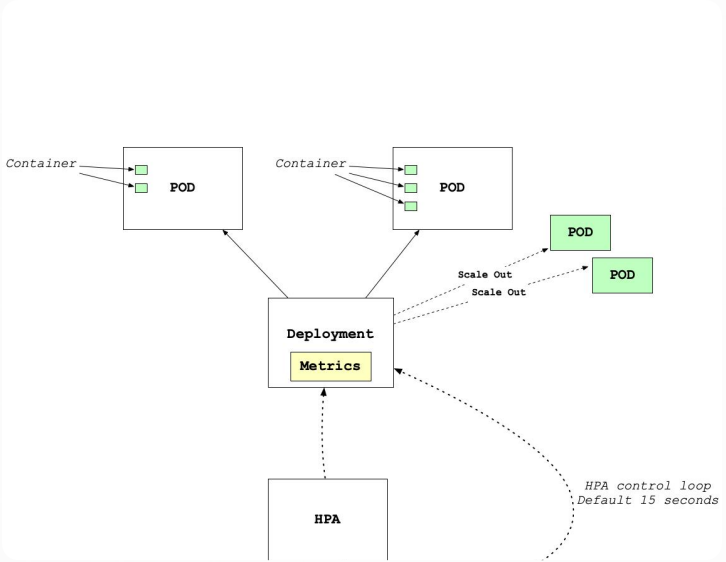
持续监控指标：实时追踪 Deployment 中 Pod 的 CPU/内存使用率，作为决策依据。

自动触发扩容：当平均使用率超过阈值（如70%）时，动态增加 Pod 副本数以应对流量。

核心配置 (YAML Snippet)

```
autoscaling:
min: 1 | max: 10 | targetCPU: 70%
```

配置解读：服务保持最少1个实例，峰值自动扩容至10个，利用率低于阈值时自动收缩，最大化资源利用率。



Kubernetes HPA 自动扩缩容流程示意

HPA Controller Loop (Default 15s)

6. 项目结构与代码实现

入口定义层 (Cmd & API)

`cmd` 存放项目启动入口 `main.go`；`api` 目录集中管理 CRD 资源定义与 API 接口规范，实现业务模型与代码的解耦。

核心业务逻辑 (Internal)

`controller` 实现控制器核心协调逻辑；`resources` 封装 Deployment/Service 等 K8s 资源构建工具，大幅提升代码复用性。

部署配置管理 (Config)

集中存放 CRD 安装、RBAC 权限及控制器部署 YAML 模板，支持环境隔离与灵活的运维配置。

kubeai-platform / src

```
├──cmd/# 程序主入口
│  └── inference-gateway/
│  └── main.go
├──api/# CRD 与 API 定义
│  └── v1/
│  └── inferenceservice_types.go
├──internal/# 内部核心逻辑
│  ├── controller/ # 协调控制逻辑
│  └── resources/ # K8s资源构建工具
└──config/# 部署与运维配置
```

6. 代码实现：Controller 核心逻辑

`inferenceservice_controller.go` 是控制器的核心实现文件，通过 `Reconcile` 函数完成从资源获取、中间件协调到状态更新的完整闭环。

```
// Reconcile 核心协调逻辑：状态驱动的资源编排
func (r *InferenceServiceReconciler) Reconcile(ctx context.Context, req ctrl.Request) (ctrl.Result, error) {
// 1. 获取 CRD 实例，作为协调的数据源
isvc := &aiv1.InferenceService{}
if err := r.Get(ctx, req.NamespacedName, isvc); err != nil { return ctrl.Result{}, client.IgnoreNotFound(err) }
// 2. 协调核心资源：Deployment (Pod管理) & Service (流量入口)
if err := r.reconcileDeployment(ctx, isvc, resources.NewStableDeployment(isvc)); err != nil { return
ctrl.Result{}, err }
if err := r.reconcileService(ctx, isvc, resources.NewStableService(isvc)); err != nil { return ctrl.Result{},
err }
// 3. 扩展逻辑：灰度路由 / Ingress 配置 / HPA 自动扩缩容 ...
if err := r.updateStatus(ctx, isvc); err != nil { return ctrl.Result{}, err } // 4. 更新资源状态
return ctrl.Result{}, nil// 协调成功，等待下一次事件
}
```

6. 代码实现：资源构建工具

`internal/resources/` 目录下的工具函数负责将 `InferenceServiceSpec` 配置声明，转换为具体的、可被 Kubernetes 集群调度的资源对象（如 Deployment, Service 等）。

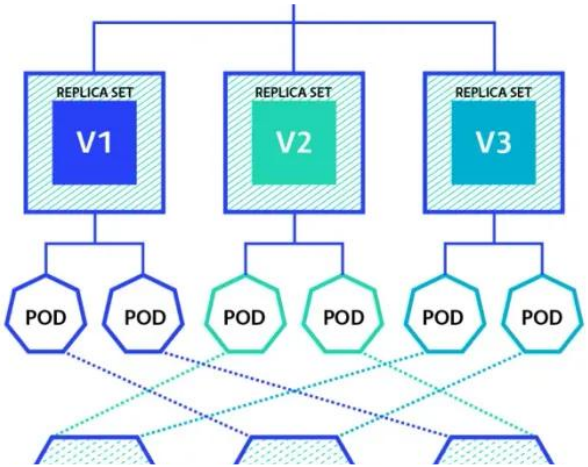
核心逻辑：Deployment 构建 (deployment.go)

```
}
return&appsv1.Deployment{
ObjectMeta: metav1.ObjectMeta{
Name: isvc.Name +"-stable", Namespace:
isvc.Namespace,
Labels: labels,// 标签与控制器关联



OwnerReferences:
[]metav1.OwnerReference{*metav1.NewControllerRef(
```

架构映射：从逻辑到物理资源



7. 部署与测试

01. 安装 CRD 资源

将自定义资源定义 (CustomResourceDefinition) 安装到 Kubernetes 集群中，为后续部署做准备。

```
$ make install
```

02. 部署 Controller

将核心控制器以 Deployment 的形式部署到集群中，控制器将负责监听并处理 InferenceService 资源。

```
$ make deploy
```

创建 InferenceService 服务实例

```
apiVersion:ai.kubeai.io/v1
kind:InferenceService
metadata:
name: bert-classifier
spec:
modelName: bert-classifier # 模型名称
replicas: 2 # 初始副本数
resources: { requests: { cpu: 1, memory: 2Gi } }
autoscaling: # 自动扩缩容配置
minReplicas: 1, maxReplicas: 10
canary: enabled: true # 开启金丝雀发布
service: port: 80, host: bert.example.com
```

7. 验证结果

查看服务状态 (InferenceService)

```
kubectl get inferenceservice
# 输出: NAME READY URL AGE
```

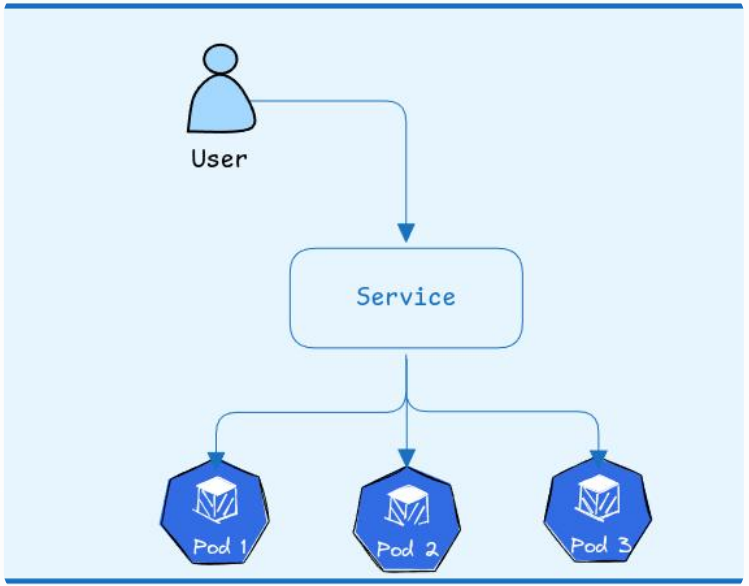
查看底层相关资源

```
kubectl get deploy -l app=bert-classifier
kubectl get svc -l app=bert-classifier
kubectl get ingress
```

CURL 测试服务访问

```
curl
http://bert.example.com/v1/models/bert:predict
```

Kubernetes Service 资源拓扑



8. 总结与展望

核心总结



标准化建设

提供统一的模型服务部署和全生命周期管理标准。



全流程自动化

自动化处理K8s资源的创建、更新与维护工作。



高可用架构

支持灰度发布与弹性伸缩，大幅提升服务稳定性。



极致易用性

对用户屏蔽底层复杂的K8s操作，降低使用门槛。

未来展望



引入服务网格

集成Istio实现基于用户、地域等更复杂的流量路由策略。



多模型服务

支持在单个服务中部署多个模型，实现高效的多路推理。



智能模型预热

Pod启动时提前加载模型，有效减少首条请求的响应延迟。



全链路监控

集成Prometheus/Grafana，提供丰富的指标与可视化面板。

Q&A

感谢聆听

THANKS FOR LISTENING

豆包AI生成